LINMA2472 : Algorithms in Data Science

Homework 2 : CGAN to generate the best letters

Students :

Delneste Jeanne 26491900

Devillet Maxime 08661900

Terryn Eleonore 66511900

November 2023

I Train your CGAN on MNIST for image generation

I.1 Your CGAN model

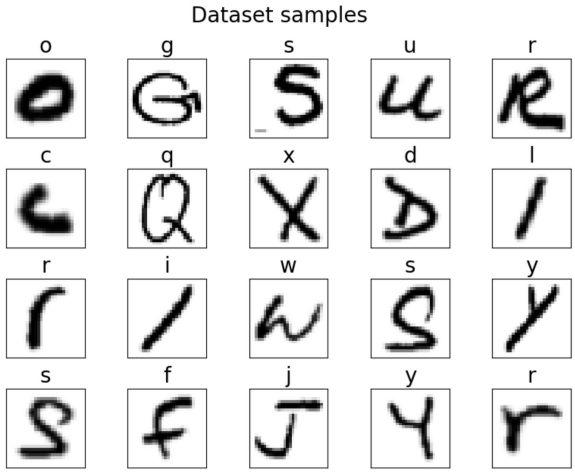
Generator model We didn't change a lot the generator model. The only change we made is to set the parameter `ngf` à 100, which is the depth of feature maps in the last layer of the generator. So our generator model takes as input a batch of latent vector of dimension 50 drawn from a standard normal distribution $N(0, I_{dim_latent_space})$ and returns a 1x28x28 pixels image. So the dimension of the latent space for the generator is kept as 50. As we kept the `batch_size` to 64, the generator receives actually 64 vectors at once and returns 64 images.
As we see using `getInfoModel` there are a total of $2,369,765$ parameters in the generator model and a total of 15 layers. There are a mix of one Embedding layer, one Linear, 4 BatchNorm2d, 4 ReLU, 4 ConvTranspose2d and a Tanh layer.

Discriminator model To find the best CGAN model we tried several things based on the Reference model that was given to us. We tried to vary the LeakyRelu parameter in the Discriminator to see what was his influence, we tried reducing the number of layers and we also tried with removing the Dropout layer. We also tried to just set the `ndf` parameter to 100. With all these results we decided to keep the reference discriminator model with `ndf` set to 100 because it was the one who had the best convergence.
The discriminator model takes as input a batch of 1x28x28 pixels images and returns for each image a scalar probability between 0 and 1 representing that the image came from training data (1) rather than the generator (0). As before the batch size is 64 so it receives 64 images at once and returns 64 probabilities.
As we see using `getInfoModel` there are a total of $254,081$ parameters in the generator model and a total of 11 layers. There are a mix of one Embedding layer, 2ne Linear, 4 Conv2d and 4 LeakyReLU layers.

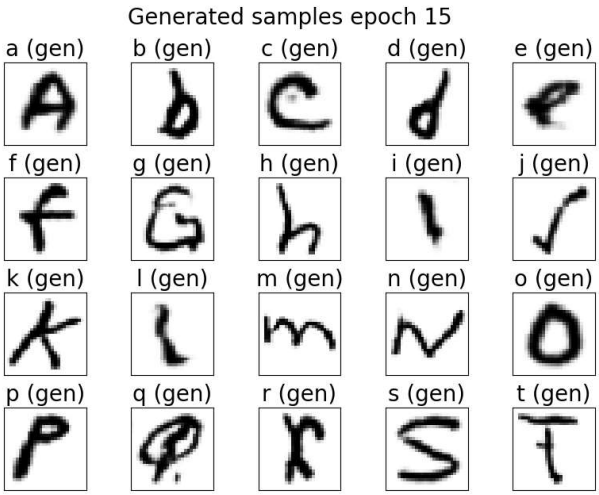
I.2 Training your CGAN

We trained our CGAN model on the EMNIST Letters data (as it was asked in the Notebook) with 15 epochs. We used Google Collab with a T4 GPU, the Adam algorithm where we change the parameters of this optimizer : a learning rate of 0.0004 and betas of (0.25, 0.9). As said before, the batch size used is still 64.

Below, in (Fig1), you see the results we get :



(a) Sample from the EMNIST_Letters dataset



(b) Generated images by our generator

FIGURE 1 – Visual comparison between the real and fake images

We see that our results are pretty good, the images look like real letters.

I.3 Error metrics during training

I.3.1 Accuracy

The accuracy here refers to the capacity of the discriminator to correctly recognize the real and fake images. It is computed as taking the ratio between the correctly classified samples and the total number of samples.
With the function `computeGANAccuracy`, we can compute the accuracy over the real samples, the fake samples, and the overall accuracy. On the graph below, we can see all these accuracies for the training set and the test set.

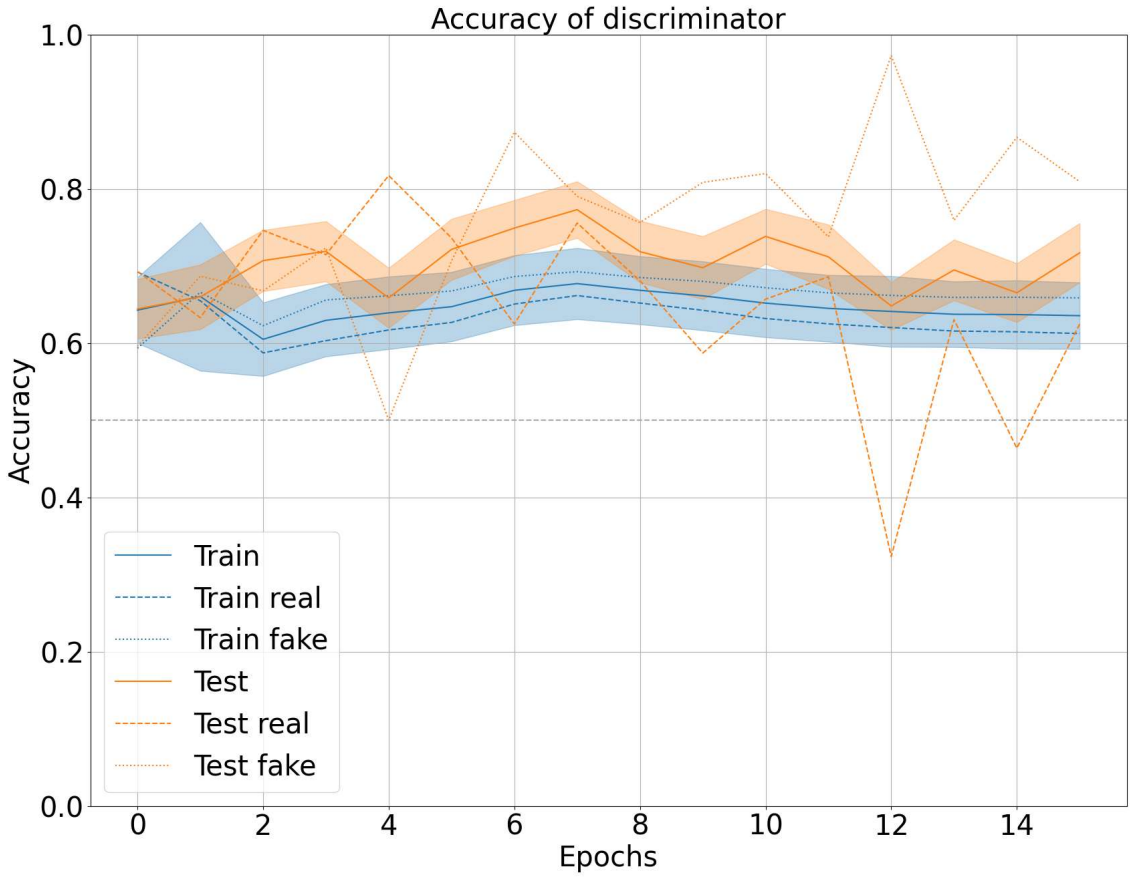


FIGURE 2 – Accuracy of the discriminator

We can observe that for all sets, the accuracy stays consistent and above 0.5, which is pretty good. This means that our discriminator can correctly classify the figure in more than 50% of the cases.

I.3.2 Loss

In general, the loss function computes the difference between the predicted and the actual data. There exists several ways to compute the loss function, but we can always say that the generator tries to minimize it, while the discriminator tries to maximize it. The loss function used here is the `BCEWithLogitsLoss` from the PyTorch library. "*This loss combines a Sigmoid layer and the BCELoss in one single class. This version is more numerically stable than using a plain Sigmoid followed by a BCELoss as, by combining the operations into one layer, we take advantage of the log-sum-exp trick for numerical stability.*" [los]

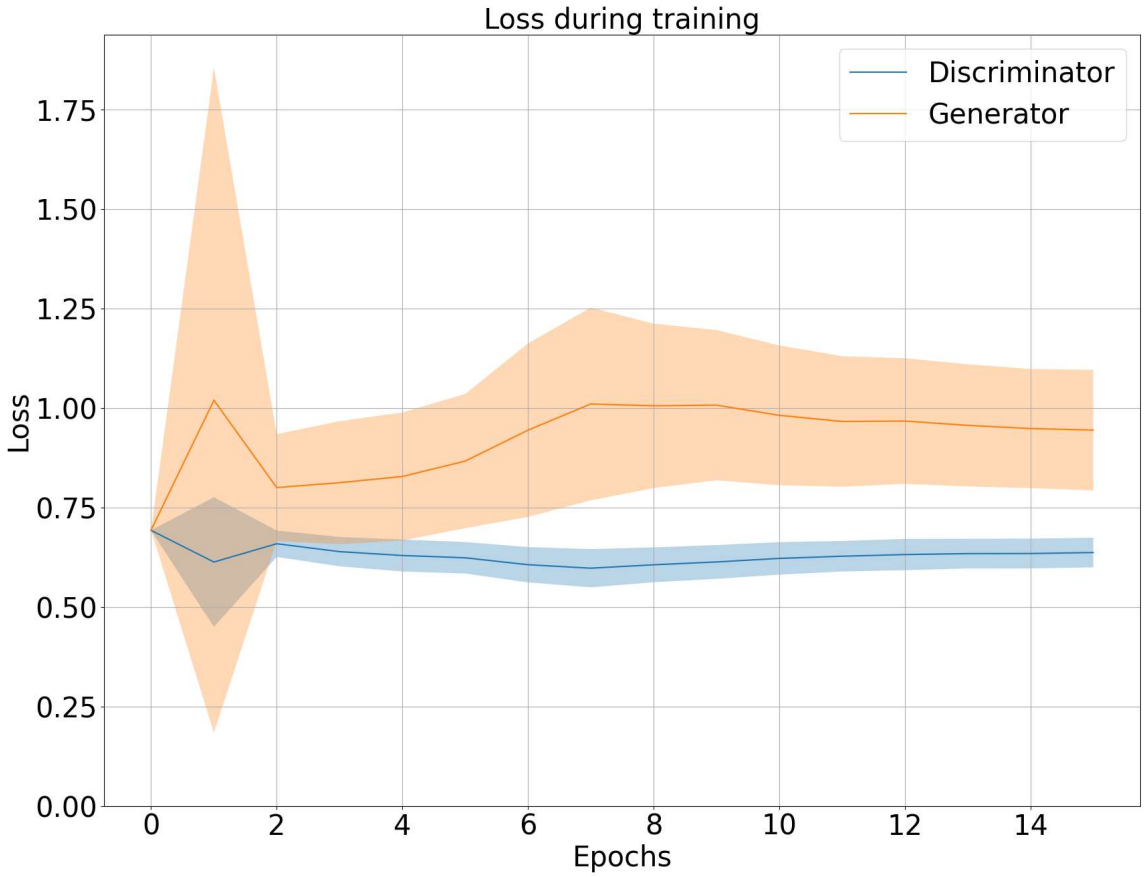


FIGURE 3 – Loss function

We can see that the relative values of the discriminator and the genrator are balanced. Neither the generator or the discriminator dominates the other, which is good.

I.4 Improve your CGAN model

I.4.1 How we have improve the reference model

— **ndf and ngf** As mentioned earlier, the technique we used to significantly improve our GAN was to increase the size of the feature maps' depth (ndf and ngf) to make the model more complex and better suited to handle the intricacies of the problem by expanding its capacity. Increasing ndf from 32 to 100 and ngf from 16 to 50, for instance, allowed us to increase the model's parameter count from 245,000 to over 2.3 million, a tenfold increase. While increasing the model size can lead to better results, it's crucial to note that the computation time for each epoch also increases. In our case, the computation time per epoch doubled. Additionally, if a model is too large for the given problem, it may struggle to converge and could face overfitting issues. Therefore, it's essential to conduct numerous tests to find the size that is the most suitable for our specific problem.

— **Learning rate and beta** Next, our second improvement focused on adjusting the learning rates (lr-discriminator, lr-generator), increasing them from 0.0002 to 0.0004. The learning rate determines how quickly the model learns. A higher learning rate means the model adapts its learning and parameter adjustments more swiftly. Since we increased the number of parameters, raising the learning rate allowed the model to learn more quickly, avoiding prolonged waiting times for results.

However, it's important to note that an excessively high learning rate can lead to instability, preventing the model from converging as it oscillates between extremes in an attempt to correct itself. To address this, we modified the beta, which gradually decreases the learning rate as the model learns. This approach allows the model to initially progress rapidly and later proceed more slowly and cautiously. We adjusted beta from (0.5, 0.999) to (0.25, 0.9) for both the discriminator and the generator, ensuring a smoother learning process.

I.4.2 What we have tried

While we did find the parameters that improved our results, we also experimented with some that yielded less favorable outcomes, even causing the model to perform poorly. Here are a few ideas we tested that did not deliver the expected results in their current form :

— **Adding and removing convolution layers**

1. **Adding a layer** Adding a layer increases complexity and enables capturing finer patterns by introducing additional computations to the parameters. However, in our case, adding a layer yielded poorer results and prevented the model from converging, with a significant fluctuation in the test-train performance from one epoch to another.
2. **Removing a layer** Next, we attempted to remove a layer to prevent overfitting and aid in faster convergence towards a solution. However, even this time, the results were inconclusive. Indeed, the model converged very quickly, and the results did not vary much from one epoch to another. This ultimately led to an acceptable but less satisfactory outcome than the reference model.

In the end, we concluded that the original model's number of layers was the most effective, and we opted to explore improvements through other avenues.

— **Give avantage to the generator** During our analysis, we noticed that the discriminator tended to train faster than the generator. Therefore, we experimented by giving an advantage of 1.3 or even 10 times to the generator compared to the discriminator. We observed better results, especially with the factor of 3, which visually appeared more promising. However, when we attempted to increase the model's complexity with ndf, the generator's advantage became more of a burden and led to poorer outcomes, causing overfitting of the generator to the discrimi-

nator. In tests, this resulted in the generator consistently outputting 100% true or 100% false. Consequently, we decided not to use this approach in the final version.

— **Changing the optimizer algorithm** AdamW is an improvement over Adam, addressing certain limitations of the Adam algorithm. We decided to switch algorithms, expecting improvements, but interestingly, we observed no significant difference regardless of the algorithm tested (AdamW, ReLU, normalization, etc.). Consequently, we stuck with the original version that performed exceptionally well.

In the end, we tested numerous parameters and various combinations of parameter changes ; however, very few yielded better results than the reference model. Throughout our analysis, we explored and abandoned numerous hypotheses, leaving most parameters in their original configuration. However, we acknowledge that even if a particular test didn't work, it doesn't mean it won't work with a different combination or a different dataset. Sometimes, a minor adjustment can entirely change the final outcome.

Based on our observations, the combination we chose appears to deliver more than satisfactory results and stands out as the best option in our assessment.

I.4.3 Ideas and researchs

These are 3 ways to improve the model :

1. First way to improve the model would be to incrase the `num_epochs`, the number of training epochs to run. The more you train your model the better it gets but it would take longer to run. [pyt] It already takes more than 50 min to run our code. Additionally, if the model isn't flexible enough or the loss function eventually stops improving, it means the model has reached its learning limits. In such cases, continuing to train the model for more epochs may not lead to significant improvement. Therefore, it's crucial to carefully choose the number of epochs based on how quickly you observe the model stops making progress. This thoughtful adjustment is essential to maximize the model's performance without unnecessary computational overhead.
2. An other way that could improve our model would be to use the minibatch discrimination as it is told in [TS16]. The concept of it is quite simple : any discriminator model that looks at multiple examples in combination, rather than in isolation, could potentially help avoid collapse of the generator. In this document, and below (Fig4), we see that the samples generated with this discrimination are really better and indistinguishable from the dataset images.

FIGURE 4 – (Left) Samples generated by model during semi-supervised training. (Right)Samples generated with minibatch discrimination

It would be a good improvement for our model because our generated samples are not as visually good as the one we see in Fig4.

3. The last way of improving the model would be to use the AdamW optimizer instead of Adam (in a better way than what we have tried). As it is told in [Ada], AdamW is Adam with weight decay in which the weight decay is performed only after controlling the parameterwise step size. Theoritically, it is also told that AdamW yields better training loss and that the models generalize much better than models trained with Adam. Maybe when we tried using AdamW algorithm we didn't do it correctly or with the good parameters.

II Explore the latent space

Because of the uncertainty of our model and a timing problem, we had to work with the reference model for this part.

II.1 Diversity in each class

We tried to generate several times the same class, first with a random latent space, and then with the mean latent space, i.e. the vector filled with zeros. You can see below the result with random latent space vector, and a scaling of 3. We observed that the larger the scaling factor, the more the model varied significantly. However, it tended to disperse and no longer resemble the initial letter at all. Indeed, the scaling factor takes a randomly generated vector from the embedding function and multiplies its values, allowing for a larger range of randomness. One way to explain this observation is that the AI was initially trained with a scaling factor of 1, and thus, it encounters extreme data that does not resemble what it has learned. This could explain such results.

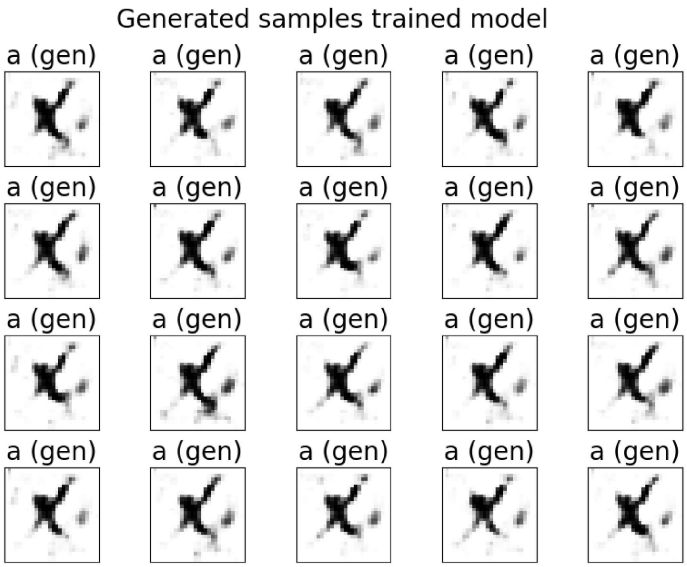


FIGURE 5 – Diversity of the letter a with random latent space vector

And here with a latent space vector of zeros.

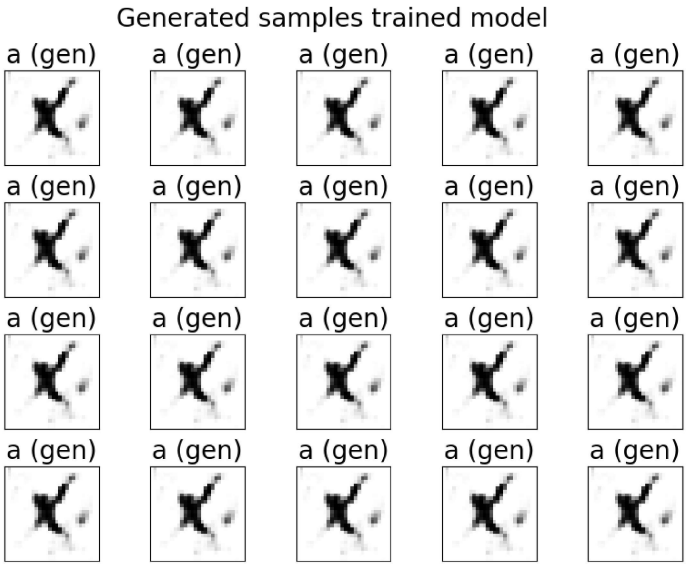


FIGURE 6 – Diversity of the letter a with zero latent space vector

We can observe here that with the letter a (and the other letter, but we do not show them here), the outputs are not very diversify. This means that the generator is not very efficient, because we want a generator that can produce different letter a so that it will fool the discriminator. The more the generator is diversify, the more it is difficult for the discriminator to classify the image.

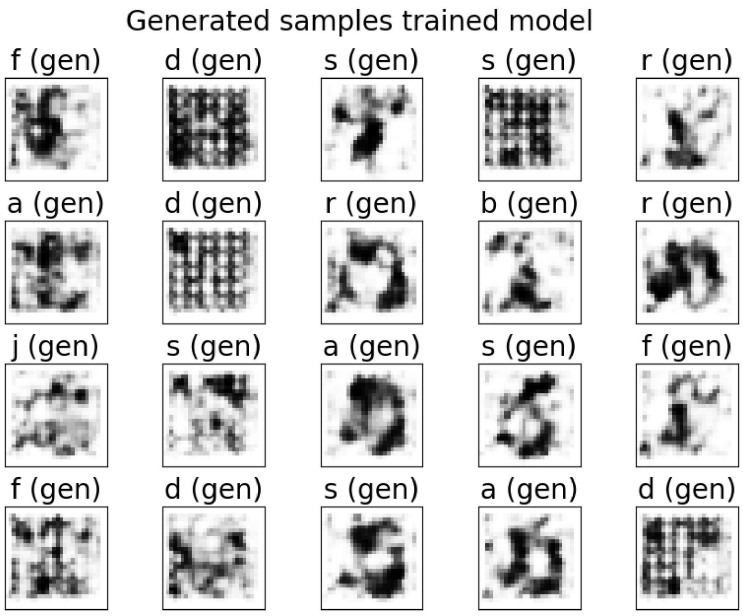
The difference is slight but we can see that with a latent space of zeros, the images are even less diversify.

II.2 Discrimination on each class

We want to observe here if the discriminator is better for one class in particular. A good discriminator is a discriminator that is fair for all the classes, which means that can correctly detect the true and fake images with the same accuracy for all the classes.
We tried to generate a lot of samples of one class with our generator, and then to take the same amount of data from the dataset of the same class. Then we computed the accuracy of the discriminator over these samples. But we had a problem in the computation of the accuracy.

II.3 Outside of distribution

We want to discover what happens if you try to generate points with labels that are not in the training set. Here is what we obtain.

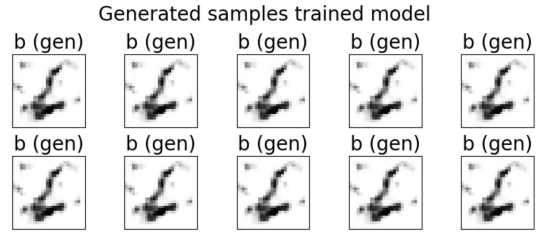


We can obviously see that the generated figures do not look like letters symbols at all. They do not make sense visually, this could be because of our lack of diversity. Another explanation could be because the generated labels are totally random, the model will be lost and will want to stick to the nearest embedding's label. This could explain why we can guess the letter in the pixel cloud on the figures.

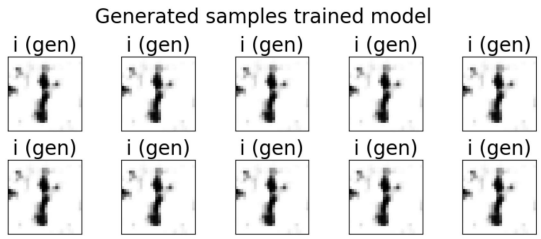
II.4 Interpolation for exploration

II.4.1 Within the same class

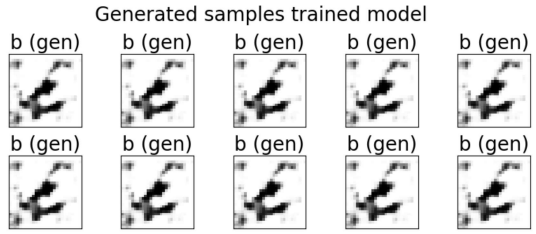
We generated 2 random latent space vectors, and then we computed their interpolation. Here is what we obtained :



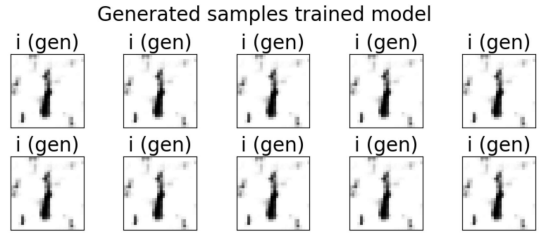
(a) `b` - latent space vector 1



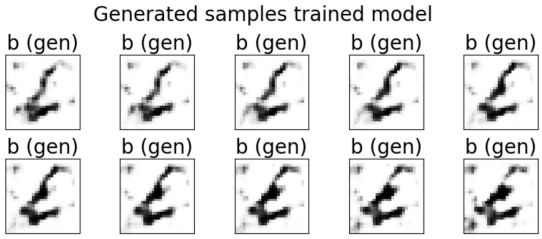
(b) `i` - latent space vector 1



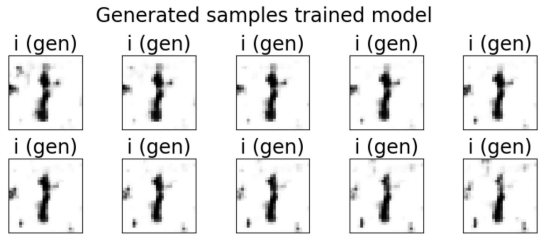
(c) `b` - latent space vector 2



(d) `i` - latent space vector 2



(e) `b` - interpolate latent space vector



(f) `i` - interpolate latent space vector

FIGURE 7 – Interpolation

The first figures are with the first latent space vector, the seconds with the second latent space vector, and the lasts images are the result of the interpolation of the two latent space vectors. We can observe on the interpolate images that, at first, the image looks like the ones from the first latent space vector, and progressively with the iterations the images tend to the ones from the second latent space vector.

Références

[Ada] Why adamw matters. `https://towardsdatascience.com/why-adamw-matters-736223f31b5d`. [Online ; accessed 21-November-2023].

[los] Bcewithlogitsloss. https://pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html#bcewithlogitsloss. [Online ; accessed 22-November-2023].

[pyt] Dcgan tutorial. https://pytorch.org/tutorials/beginner/dcgan_faces_tutorial.html. [Online ; accessed 21-November-2023].

[TS16] Wojciech Zaremba Vicki Cheung Alec Radford Xi Chen Tim Salimans, Ian Goodfellow. Improved techniques for training gans. https://arxiv.org/pdf/1606.03498.pdf, 2016. [Online ; accessed 21-November-2023].